

Topics
Overview
Quick tips
Brand attributes
Applying voice and tone
Who we're writing for
Content examples • By audience ◦ Students ◦ Developers ◦ Enterprises • By medium ◦ Blog ◦ Organic social ◦ Paid social and display ads ◦ Website and landing pages ◦ Email
Grammar and punctuation
Accessibility

OVERVIEW

Welcome to the GitHub Voice Guide. Here you'll find the resources you need to write and speak for GitHub.

Why do we have voice guidelines?

1. Make everyone content creators

Guidelines and templates encourage all teams to write consistently. Most content creators don't have "content" in their job title and that's great! Guides help everyone write for GitHub.

2. Speak to people like people

This is not just about brand building—recognizing and avoiding jargon and internal language helps create readable, accessible content (yes, accessible!). Even the most technical readers or

the highest level executives want to spend as little time understanding a piece of content as possible.

3. Create one brand experience

Lots of teams write for GitHub. We have plenty of internal divisions of roles and responsibilities. But to our audience, it's all GitHub. Guidelines help us create a more consistent brand experience for our entire audience.

QUICK GUIDE

Quick guide

- ☑ **Consider your audience**

Know who you're talking to and what makes them unique.

- ☑ **Check the context**

Keep in mind where your audience will be reading your content.

- ☑ **Be conversational**

Talk like a human, not a corporation.

- ☑ **Think global**

Avoid jokes or turns of phrase that aren't globally understood.

- ☑ **Put developers first**

Start by asking, "How does this benefit developers?"

- ☑ **Default to less over more**

Write, then delete half.

- ☑ **Use correct grammar and punctuation**

Check your work for missing periods, misspellings, etc., before publishing.

BRAND ATTRIBUTES

Brand attributes

Our voice is an extension of our brand and the foundation of everything we write. To write as the voice of GitHub it's important to start from what our core brand attributes (i.e. traits, characteristics) are.

Brand attributes

When something looks and feels GitHubby to you, you're experiencing one or more of these brand attributes. Not everything we create has to have all five of those attributes reflected. They should flex up and down based on the audience, the topic, and the medium.

For more on our brand attributes, [visit the Brand Guide](#).

1. **Authentic:** We are true to ourselves, our products, and our community above all else.
2. **Nerdy:** We're excited about all things code, software, and technology, and how it intersects with life beyond devices.
3. **Empathetic:** We listen to our developers and customers first and don't assume that what's best for GitHub is best for them.
4. **Imaginative:** We go beyond what's expected to bring soul and magic into the experiences people have with GitHub.
5. **Confident:** We're experts in developer tools. And we believe in the power of our products to make developers' and business leaders' lives better.

VOICE AND TONE

Turning brand attributes into voice and tone

When it comes to writing, we convey our brand attributes using the appropriate **voice** and **tone**. Here's how this looks in practice.

Brand attribute: Authentic

Application: Talk like a person

- Be conversational.
- Use short, plainspoken sentences.
- Avoid unnecessary jargon and marketing language.

Brand attribute: Nerdy**Application: Tell technical stories**

This one can be tricky, so here's a little more context: We have our thing, and we love it. You have your things, and we love that too! Most importantly, we aren't afraid to show our passion. In fact, dialing in on the details gets us even more excited.

- Be precise. Use exact language and punctuation.
- Embrace technical language—bonus points for making it even more approachable as you go.
- Highlight interesting and expert voices—from influencers to hobbyists—whenever possible.
- Celebrate the big and small—whether it's training an algorithm to play Atari or creating programs that visualize black holes.

Brand attribute: Empathetic**Application: Consider context**

- Always consider accessibility and optimize for readability.
- Focus on your reader—use “you” more than “we”.
- Use words and descriptions that help people accomplish something.

Brand attribute: Imaginative**Application: Show optimism and soul**

- Be aspirational and share the bigger picture.
- Capture the creative soul of developers.
- Encourage dreams for a better, different future.

Brand attribute: Confident**Application: Be a resource for our community**

- Use clear, concise language that shows expertise.
- Speak with authority when presenting solutions
- Explain problems and solutions with technical expertise.

AUDIENCES

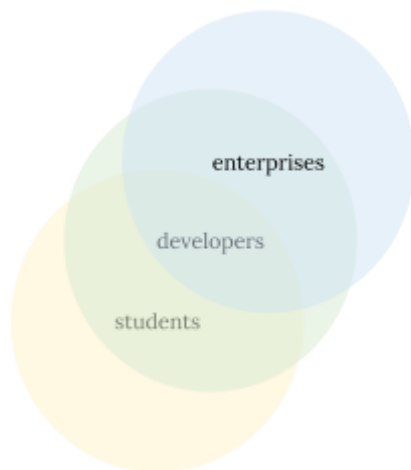
Our key audiences

We want to put brand attributes into action with the right voice and tone, whatever we say and do. But how we do it looks a little different depending on which of our audiences we're speaking to. For a complete breakdown of our audience personas, [visit the customer research repo](#).

At GitHub, we support three key audiences:

1. **Developers** who write code for work and open source, independently or as part of a cross-functional team
2. **Students** who are learning to code in traditional or non-traditional ways.
3. **Enterprise** developers and decision makers who are responsible for delivering on strategy and vision.

While these audiences may sound separate with separate goals, GitHub is a global platform. People are complex and don't always fall into one bucket or thing. For example, the after-hours open source contributor might work for an enterprise security team by day. They also might not fall into any of the buckets you see here, and that's okay—we use three buckets since these represent our major groups.



Since our audiences overlap, it's important to consider that something you're writing for one audience may reach all of our audiences. A social media post announcing an enterprise customer event is also likely to be seen by an open source maintainer. While the post may not be relevant to the open source maintainer, it should still sound like the GitHub they know and love.

EXAMPLES

Putting it into practice

Now it's time for the pen to hit the paper. Let's look at how voice and tone can change depending on *who* we're writing for.

Examples by audience

Students

For students, take a step back and examine which technical terms they know and which will have them feeling left behind. At the same time, it's important for them to feel like they can rely on expertise, without being intimidated, and feel inspired by what's possible.

Tone:

-  Dial up the imagination and excitement for what they'll achieve
-  Pause overly technical jargon

Dos and Don'ts:

-  Use encouraging and reassuring language
-  Be accessible—students come from all over the world
-  Define terms and spell out acronyms to avoid alienating new developers
-  Be a source of learning and support
-  Don't assume students know GitHub
-  Don't get too academic—teachers and professors read our content, but leave the academic tone and footnotes to them

 Works well!	 Could be better
GitHub Student Developer Pack Learn to ship software like a pro. There's no substitute for hands-on experience. But for most students, real world tools can be cost-prohibitive. That's why we created the GitHub Student Developer Pack with some of our partners and friends: to give students free access to the best developer tools in one place so they can learn by doing. Why this works: - Accessible and approachable for someone new to GitHub - Warm and welcoming the students in with use of “our friends” “free access to the best developer tools”	

- Addresses a real hurdle for new developers just starting out, and then a solution

Improve student success and increase teacher time with autograding

 Courtney Hsing

Computer science instructors are increasingly adding autograding tools to help meet the demands of teaching large classes. Autograding tools can save teachers time by providing immediate feedback to students and by automating parts of the grading process. And recent studies also highlight the benefits students perceive when these tools are used in the classroom ^[1]. Professors Dan Wallach and Mike Zamansky shared their perspectives on how autograding tools help shape student success.



About Dan Wallach and Mike Zamansky

Wallach teaches a large course as part of Rice University's introduction to computer science. Second-year and third-year undergraduates make up most of his course of over a hundred students. Wallach implemented his own autograder, [RiceChecks](#), which builds on the features of the Gradle build-automation tool.

Zamansky teaches a computer science lab that pairs with CS1 at Hunter College. Last year in his course of first-year undergraduate students, he implemented Travis CI, a service that automatically generates builds and runs tests.

Time for more meaningful feedback

For teachers, autograding tools free up time to provide more meaningful feedback on aspects of student work that cannot be assessed by automation. For example, Zamansky used the extra time to focus on parts of the code that revealed "something about the students." He went on to state that an English teacher would never use an autograder on an essay because they have to read the students' work in order to get to know them. However, they can rely on automated grammar checks. By using an autograder, English teachers can automate away the tedious parts of grading and free up time to focus on the elements of writing that tell them more about the student.

Wallach offered the same perspective and encouraged teachers to automate whenever possible in order to "free up your human graders to look at the things you can't automate." Automating the more tedious parts of grading refocuses teachers' grading efforts on more complex elements of a student's work, and contributes to their success by helping teachers get to know their students.

What could be better:

- Post feels more academic than conversational and accessible
- Not clear whether this is for students or teachers

	- While headline focuses on the benefit, body copy makes the “how” hard to find
--	---

Developers

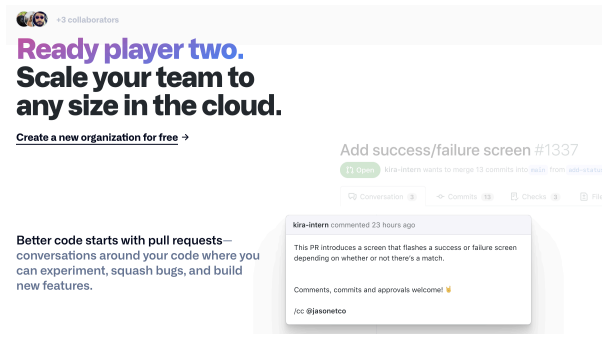
For developers, scale back the whimsy and blue sky and share more practical guides and how-to's. Our expertise is important but they're also looking for new perspectives, learning opportunities, and features that can make a difference in their day-to-day. And when something goes wrong, they appreciate timeliness and transparency over empty reassurance or platitudes.

Tone:

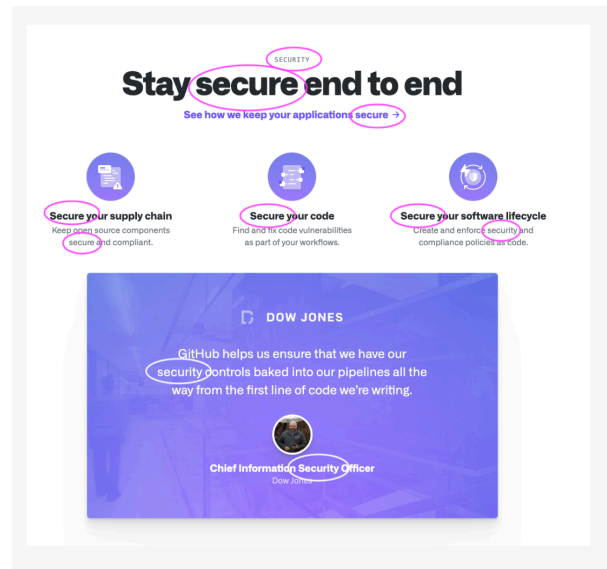
-  Dial up the technical details and authenticity
-  Pause marketing language and unnecessary jargon

Dos and Don'ts:

-  Use practical, technical language
-  Be precise and accurate in word choice and grammar
-  Ensure every sentence adds value
-  Add personality with relevant, relatable references and examples
-  Don't define common acronyms, like HTML or CI/CD
-  Don't use corporate wesppeak (the GitHub family, sophisticated solutions, paradigm shift)

 Works well!  Why this works:	 Could be better
---	--

- Speaks immediately to developer benefit
- Leans into the nerdiness with a play on words developers will recognize
- Shows specific examples of what developers can expect to see from the product



What could be better:

- Multiple uses of “secure” and “security” are redundant and repetitive
- Subheads don’t add new value
- Doesn’t show how GitHub can improve their experience (ex: save time by securing code early, automate security scanning, etc.)
- Doesn’t do enough to differentiate GitHub features from similar tools they might already use

Enterprises

GitHub is made for developers. They’re our core audience—and our biggest advocates. When we’re talking to enterprises, we’re also talking to the hundreds (even thousands!) of individual developers who make up these organizations. Yes, we care about innovation. Of course we care about speed to market. But most of all, we care about developers, the people who build the software. And if your organization is only as successful as your software, then those people matter more than anything.

This is how we show our value to engineering managers, VPs of engineering, and CTOs. These leaders may not use GitHub every day, but they work with the developers who do. They could also be developers themselves or were in the past. So while we might not get into all the technical details, we still want to make it clear we know what we're talking about. Enterprise leaders want to know we speak their language, understand their business, and have clear ideas on how to push it forward through software. We prove it by putting developers first in everything we do. When we talk about how to solve challenges like automation, security, and developer productivity, we always ground it in how it makes their developers' lives better. That means writing for enterprises shouldn't feel like speaking to an entirely separate audience. It should feel like a natural extension of our developer brand.

Tone:

-  Dial up the empathy and confidence
-  Pause overly technical details

Dos and Don'ts:

-  Speak to the challenges software teams face
 -  Be a source of inspiration and optimism, reframing challenges as solutions
 -  Set the vision, but don't forget the journey it takes to get there
 -  Stay true to our developers and community
 -  Choose the right enterprise audience—people with different job titles (Engineering Managers, Security Engineers, CTOs, etc) have different needs
 -  Treat enterprise buzzwords like salt—it's okay to sprinkle in words like DevOps, innovation, and transformation that leaders are looking for, but too much ruins the meal (err, copy)
-
-  Don't be too playful around more serious subject matters like security
 -  Don't lose a developer-first mindset
 -  Don't prioritize quantity over quality (ensure every line is critical and easily scannable for busy readers)

 Works well!	 Could be better
---	---

October 7, 2020 — Enterprise

What is DevOps? A guide to common methods and misconceptions

Jared Murrell

By now, most people in technology are familiar with the term DevOps. What we call "DevOps" will often differ between organizations, yet one thing remains the same: DevOps is defined by people building software and how they work together, not simply by what's in their toolchain.

As part of GitHub's Professional Services team, I've had the opportunity to see the way many companies have adopted DevOps and what it looks like for teams in their day-to-day work. Here's what we've learned—and some common DevOps misconceptions we've encountered along the way.

What is DevOps?

DevOps has been defined in many ways: a set of practices that automate and integrate processes so teams can build, test, and release software faster and more reliably; a combination of culture and tools that enable organizations to ship software at a higher velocity; a culture, a movement, or a philosophy. None of these are wrong, and they are all important aspects of DevOps—but they don't quite fully capture what's at the heart of DevOps: the essential human element between Dev and Ops teams, when collaboration bridges the gap that allows teams to ship better software, faster.

For organizations, DevOps provides value by increasing software quality and stability, and shortening lead times to production. For developers, DevOps focuses on both automation and culture—it's about how the work is done. But most importantly, DevOps is about enabling people to collaborate across roles to deliver value to end users quickly, safely, and reliably. Altogether, it's a combination of focus, means, and expected results.

Sh

tw

in



1,795,464 followers
1w •

Whether it's stopping to respond to issues or merging a pull request, the small things start to add up. Tune into next week's [#GitHubInFocus](#) for techniques and tools to cut down on manual tasks and help build your perfect workflow.
<https://github.co/3rY2Cjh>



GitHub InFocus · Virtual 2021 · Starting Feb 23

infocus.github.com • 1 min read

Why this works:

- Speaks immediately to the value of this event for developers
- Calls out specific and relatable pain points vs. sounding like broad "marketing-ese"



1,795,464 followers
1w •

GitHub InFocus is coming to you LIVE in APAC timing! Click on through for an enterprise look at the future of software—starting February 23.

[#github](#) [#InFocus](#) [#developer](#) [#devops](#) [#security](#) [#devsecops](#) [#CICD](#) [#enterprisesoftware](#)

<https://github.co/3q1dU5P>



GitHub InFocus · Virtual 2021 · Starting Feb 23

infocus.github.com • 1 min read

What could be better:

- Event tagline promotes/talks up the the event itself, not why developers and teams should attend
- Social copy is focused on signups, not the value for software teams and developers

⊘ Could be better

Empower your team. Transform your business.

DevOps is just the start. Top organizations know that transformation also depends on technology, talent, culture, and process. GitHub helps enterprises put them all to work—in one place.

What could be better:

- Uses enterprise buzzwords ("empower" or "transform") that are vague and don't communicate concrete value
- DevOps is an important topic/term for enterprises, but is buried in the subhead copy

--	--

Examples by medium

Meet your audience where they are. Different formats need a different voice and tone. Let's look at how voice and tone can change depending on *where* we're writing.

The GitHub Blog

The GitHub Blog has a few different audiences—developers, enterprises, press, and more. Most traffic to the GitHub Blog comes from social media. This means there's a lot of crossover between our social media and blog audiences. We also see traffic from Google, as it's well-indexed for search. Keep in mind, folks are looking for something from our blog: Whether that's help, how-tos, or more information about a product or feature. Make sure your posts give them what they're looking for—and make it clear why they should get it from us, instead of somewhere else.

Guidance:

Writing style

Keep posts concise and conversational. Default to writing like a friend, rather than writing like a company. Make it clear who you're writing for, especially if your topic requires a certain amount of experience to understand. Avoid any localized turns of phrase or idioms that might feel confusing or exclude our larger community.

Formatting

Keep introductions short, but don't skimp on the technical details. Include subheadings and bullets that break up paragraphs into shorter sections and make it easier to scan. No one wants to read a wall of text, so keep readers interested with eye-catching images, videos, code, and lists. For easy navigation, try adding anchor links to help folks find specific sections. Posts should also be as actionable as possible. Think about your calls to action (CTAs) and how you're driving folks to click them—if you include several CTAs versus one, folks might not be sure which one is the most important or click away before they finish.

Dos and Don'ts:

- ✓ Choose topics that teach, show, and add value
- ✓ Get into the details (including the code)
- ✓ Use contractions and friendly, helpful language
- ✓ Include clear titles and subheaders so readers can easily scan

✗ Don't bury the lede

- ❌ Don't use corporate speak (the GitHub family, sophisticated solutions, paradigm shift)
- ❌ Don't overwrite—for most purposes, your introduction doesn't need to be multiple paragraphs

 Works well! LINK Even better code review in GitHub for mobile  Ryan Nystrom GitHub for mobile lets you get a lot done without having to sit at your desk. We've heard from users who use the app to review code while taking a walk or sipping coffee on their balcony. Oh, or boating down the Amazon river. And, a lot of these people aren't just triaging notifications in the app—they're reviewing and merging code. Since launch we've seen almost 700,000 reviews, and over 350,000 merges, which is a lot of code to ship while you're on the go. We've also heard lots of feedback about ways to improve the code review experience in GitHub for mobile. So in our newest version, we're shipping a bunch of features that make code review easier, faster, and more productive. More tools for code review The new version of GitHub for mobile features an overhaul of the code review experience. For example, it can be super painful to read code on your mobile device when the lines keep wrapping. In fact, the ability to turn off line wrapping is the top feature request from the community. So we added a toggle for that!  Why this works: - Headline is very clear about what the reader can expect in the post - Written in a very human and personal way, you can get a sense for the author's personality - Focuses on the user's needs—addresses how developers use the app and how they're improving it to make it even more useful	 Could be better September 14, 2017 — Events Introducing our Universe Community Partners  Bianca Escalante With GitHub Universe one month away we are excited to announce our 2017 Community Partners! We choose Community Partners based on several criteria but the three main questions we ask ourselves when reaching out to potential organizations are: • Does their work assist in lowering barriers for people from underrepresented backgrounds to enter and succeed in the tech industry? • Do they have an audience that can benefit from complimentary tickets to the conference? • Are they making a positive social impact, namely in the geographic region where the conference will take place? GitHub's push towards a more diverse, inclusive and accessible Universe is rooted in the fact that bringing together people from disparate backgrounds fosters innovation within our industry. If we're not working to actively engage people from all walks of life, we're doing our community a disservice. The more we can bring diverse communities together, the more enriching, educational and valuable an experience we can provide for everyone. Meet our 2017 community partners at GitHub Universe           With that, we are happy to introduce you to this year's Universe Community Partners. We encourage you to read ahead in order to learn more about them and the valuable work they do. What could be better: - Post is an introduction to new partners, but the partner names (the purpose of the post) aren't shared until four paragraphs later. - Spends more time explaining our process than focusing on the partners themselves—and even more importantly how the partners help developers.
---	---

🚫 Could be better

GitHub's site policy updates are ready for your feedback

 Joe Mazzella

We regularly update our policies to reflect the evolution of our products, legal requirements, clarification requests, and user feedback. We're currently updating a number of these policies including our Privacy Statement, Subprocessors and Cookies Policy, Terms of Service, and Corporate Terms of Service. For example, we're updating our privacy policies to account for potential developments like Brexit and changes in our use of cookies and subprocessors. We're also clarifying prohibited uses of GitHub due to export controls and sanctions laws, and content restrictions related to acceptable use.

Share

 Twitter

 Facebook

 LinkedIn

The changes we're making are in newly-opened pull requests for each policy in our [Site Policy Repository](#). You can see the changes there and make comments, just like in any other pull request. This comment period starts today and ends on April 12 at 5 pm PT. We'll take a week after that to go through all of your comments and carefully consider making further improvements. Then, on April 19, we'll merge the new changes into master, and the new terms will become effective.

[Explore the updates](#)

Why we open our policies to feedback

There are two main reasons we update policies in this way. First, we want to make it easy on GitHub users—who are familiar with using pull requests to review code updates—to see exactly what has changed and understand the changes. Second, we're all used to collaborating on open source projects to make them the best they can be. By turning our policies into open source projects, we're making it easier for users to engage with our site policies.

How you can use our site policy project for your team

Our policies in the Site Policy Repository are all licensed CC0, so feel free to fork a copy to adapt for your own site. We'd love to hear if our project helped you create policy foundations for your organization, so [let us know!](#)

[Learn more](#)

What could be better:

- Title is GitHub-oriented vs. reader-oriented, "Give your feedback on GitHub's site policy updates" is more direct and makes it clear what readers will learn or what action we expect them to take
- First paragraph is all "we" instead of explaining how and why these changes benefit our users
- Most critical information in the post—how and when to give feedback—isn't included in the first few lines
- Includes multiple CTAs, and none of the CTA copy matches the action we're asking readers to take (give feedback)

	- Is multiple blocks of text without any imagery, code, pull quotes, etc.
--	---

Organic social media

Social media is where we have a chance to connect with our community in a sustained way. It's where our voice has the best opportunity to create superfans. We accomplish this by ensuring we're adding value to our audience's news feeds. We're not just competing with our competitors, we're competing with everyone that posts anything to social media. That means we don't just have to be more compelling and eye catching than our direct competitors, we have to be more compelling and eye catching than all the cat videos and spicy memes too.

Guidance:

Writing style

Our voice differs slightly platform to platform and depending on which GitHub handle we're using ([we have more than 10!](#)). The audiences for each platform and handle are different. For example, Twitter is more conversational and LinkedIn is more professional. We might dive deep on the technical details behind a new feature on @GitHub, but focus more on their solutions and benefits for teams on @GitHubEnt. But no matter who we're writing for, we want to be authentic, concise, and give folks a reason to click.

Formatting

We only have so much room (or so many characters), so we have to make it count. Don't start posts focused on what we did. Instead, lead with how it benefits the community. CTAs should be specific and encourage readers to click through—"Learn more" doesn't explain a lot or stand out, but "Sign up today!" does. With so many platforms and handles, consistency is also key. Even the way we format CTAs can make a big difference: No colons (:), but use punctuation (periods, question marks)!

Want to learn more? [Check out the GitHub social guide.](#)

See what we did there? 🐙

Dos and Don'ts:

✓ [\[Use social card templates\]](#)[\(GLOSS LINK\)](#) to create eye-catching visuals for every post (animations are best if you can create them)

✓ Consider whether your copy is adding value to the developer

- ✓ Use words that stand out but still consider the context of your followers' feeds
- ✓ Brevity is key, attention spans on social are less than those of a goldfish (for real)
- ✓ Focus on the channel, audience, and message—your copy should be where they all meet
- ✓ Be conscious about what's happening in the world, social media is often the pulse of the day's sentiment

- ✗ Don't use language targeted only to enterprises if you aren't using an enterprise handle or channel (like LinkedIn)
- ✗ Don't waste real estate with meaningless words and messages
- ✗ Don't over use hashtags, they should be purposeful to link conversations for easy discovery (e.g. #GitHubUniverse), not as a hack for growing followers
- ✗ Don't overuse emojis, they can be a great way to bring some fun to a post but can easily become distracting

✓ Works well! GitHub @github · Aug 14 Have you built your GitHub Profile README yet? Check out this video on how to build your profile, share with us, and get some inspiration from other great profiles. What will you showcase? This is a really awesome way to show off some really cool stuff on your GitHub Profile. 1:07 41.3K views 39 264 951 Why it works: - Very conversational and approachable - Assumes a certain level of knowledge that our followers would have - Opening a conversation with our followers on Twitter	✗ Could be better GitHub @github · May 2 Yesterday we rolled out a deprecation to the network graph because we are working on a new and more performant experience, and we needed to do this for technical purposes to make this succeed. 11:45 AM · May 2, 2019 · Twitter Web App View Tweet activity 24 Retweets 127 Likes GitHub @github · May 2 Replying to @github We're rolling back that deprecation so we can share more about the new experience and get more feedback. If you've been using the network graph, let us know what you think! 12 10 67 What could be better: - Isn't written succinctly or very clearly - Very flat tone - Doesn't address how this affects developers or why they should care ✗ Could be better
--	--



GitHub
@github · Sep 21

Did you know that you can now create a pull request without leaving the terminal? 🐙

› gh pr create

cli.github.com

```

$ gh pr create
Creating pull request for min-js into main in leereilly/js13k-entry
? Title Minify JavaScript
? Body <Required>
? What's next? Add metadata
? What would you like to add? Reviewers, Milestone
? Reviewers [Use arrows to move, space to select, type to filter]
> [ ] andr
> [ ] leereilly

```

0:11 80.4K views

40

886

3.2K

Why it works:

- Speaks authentically to our audience, not over explaining and talking at a higher level of technical understanding
- Assumes a certain level of knowledge that our followers would have



GitHub
@github

GitHub Satellite early bird ticket pricing is now available until April 11.

Get yours today to join software developers and influencers for technical talks and big ideas that inspire.

github.co/2HTEY4n



1:01 AM · Apr 2, 2019 · Sprout Social

View Tweet activity

5 Retweets 42 Likes

What could be better:

- Lead is all about selling tickets versus the “why” a developer should care
- Written like a sales pitch instead of sharing the value of the event for attendees

Paid social media and display ads

While paid social media and display lean more towards advertising, they’re often still presented as coming from GitHub’s social media accounts. That makes the line here tricky—we only have a few seconds to grab someone’s attention, and we’re paying for them to see us.

Guidance:

Writing style

Although we can still be friendly and approachable, the goal of paid social is to drive sign ups and generate revenue. We have to be direct here. Our message should be clear, concise, and attention-grabbing, but still stay true to GitHub’s organic social voice. Knowing our audience is critical. On LinkedIn, we can be more targeted and adopt more of an enterprise voice and terminology, since it’s understood to be a corporate networking platform and we can target who sees our ads based on their job title. But while we can also target on Twitter and Google, we should assume anyone could see our ads in any context—in an article or on their feed—and ensure our voice is approachable for our general audience.

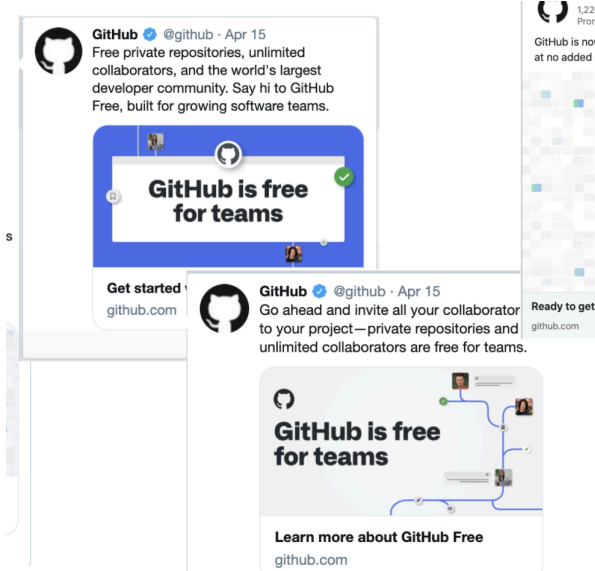
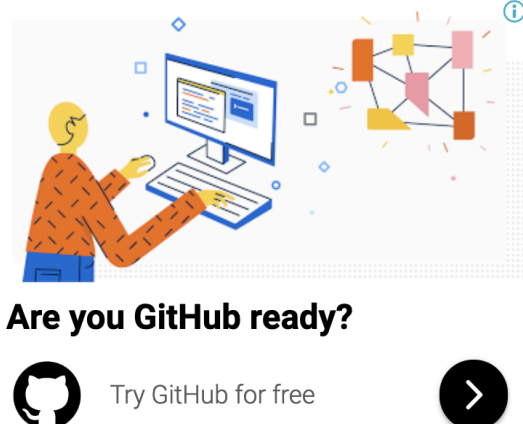
Formatting

The text copy that's above the graphic for a paid social ad is an opportunity to flex our personality, but our CTAs and image copy are what drive conversions. Every ad image should include a CTA, ideally as a button within the graphic. CTAs and image copy should also be specific and leave no question as to what we're asking viewers to understand and do. If it takes you more than five seconds to digest our ad and know what to do next, we need to try again.

Dos and Don'ts:

- ✓ Be concise in ad copy and calls to action
 - ✓ Include CTAs in images
 - ✓ Test your ads to choose the best messages
 - ✓ Stay true to our voice on organic social feeds
 - ✓ Understand your audience and their feeds
-
- ✗ Don't write copy that takes more than a few seconds to read
 - ✗ Don't assume your ads will only reach your targeted audience

Examples:

✓ Works well!  Why it works:	✗ Could be better  Are you GitHub ready? Try GitHub for free What could be better: - Assumes viewer already knows what GitHub is - Asks a question the viewer cannot answer in order to click the CTA
---	--

- Speaks immediately to how it helps the developer
- Includes clear and concise CTAs
- Still look/sounds like our social media voice

✓ Works well!



Why it works:

- Leads with the benefit for enterprise organizations
- Short and specific intro copy
- Clear CTA with the right amount of context

- No clear benefit of GitHub to the viewer

✗ Could be better



What could be better:

- No CTA or CTA button
- Mentions “enterprise,” but not clear who this is for
- No explanation of what an “open source enterprise” is or how it benefits the viewer
- “GitHub” text in lower right corner is distracting

Website and landing pages

When someone visits our site for the first time, we have about 10 seconds to convince them to stay. Like with most online content, every second and every word counts.

Guidance:

Writing style

Site pages might be longer than a social media post, but don't worry about filling up the page. Instead, think about your website like the front page of a newspaper: Eye-catching headlines and images pull readers in, while subheads and body copy provide additional information if they're interested in learning more. Focus on crafting the punchy, perfect headline and using subheads to fill in the technical details and resources.

Formatting

Copy should help guide readers through the page, giving them the information they need in the order in which they need it. It should start with the user's perspective and how they can benefit: "Get the most out of open source" vs. "We make it easy to contribute to open source."

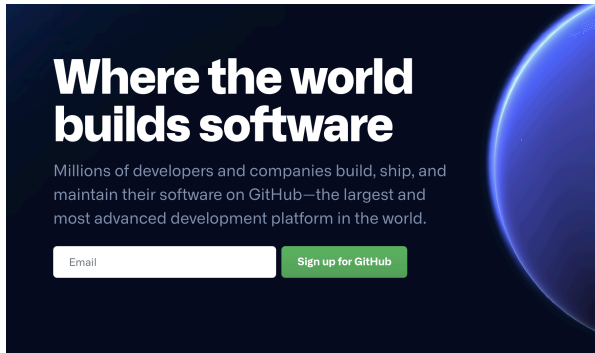
After users land on the page, our goal is to help them get to where they need to go next. Here's what this might look like.

1. *Hero (main headline at the top of the page)*: Establish what the rest of the page is going to be about.
2. *Body*: Share details in a way that builds off the section or topic before, like explaining a workflow or a process in the logical order our users would experience it.
3. *Closing*: Provide cross-promotional or related content. By now, our audience has context to dive into deeper details on their own.

Dos and Don'ts:

- ✓ Build a clear flow of information
 - ✓ Ensure information and content is scannable
 - ✓ Use active voice in all CTAs
 - ✓ Align copy and imagery
 - ✓ Recognize, validate, and address user challenges
-
- ✗ Avoid leading users off the page with unnecessary links or side topics
 - ✗ Avoid repeating phrases or concepts just to fill the space
 - ✗ Avoid writing too much over too little
 - ✗ Avoid complex turns of phrase
 - ✗ Avoid using more than three words in CTAs

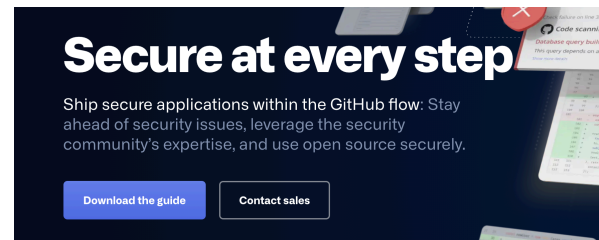
✓ Works well!



Why it works:

- Immediately communicates the purpose/reason to use GitHub
- Uses plain language, while including critical information
- Next steps and CTA are clear

✗ Could be better



What could be better:

- Assumes familiarity—what is the GitHub flow?
- Repetitive, repeats “secure and security” multiple times within headline and subhead
- Packs a lot into a small space—we’re expecting the reader to juggle multiple needs here.
- CTAs need more context—download the guide to what?

Email

When it comes to email, many of the same website dos and don'ts apply. Users scan quickly—often on their phones—and need to know immediately how and why an email is relevant to them, starting with the subject line.

Guidance:

Writing style

Whether it's a transactional email or a marketing email, stick to the most critical information and avoid the fluff. If it doesn't grab your attention, it won't grab our audience's either. Be clear about why we're emailing them—we aren't just spamming their inbox, we're providing important information about their GitHub account or the topics they care about. Keep subject lines and headers short, and put what readers need to know within the first line of text. A good best practice: write your email copy then delete 50% of it.

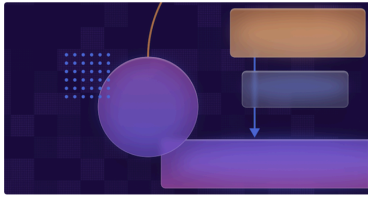
Formatting

Formatting can do a lot of the heavy lifting for us here. Instead of writing long paragraphs of text, break copy up into separate section headlines, bullets, numbered lists, etc. that are easily scannable on a small mobile screen. This is also important for plain-text emails, or emails that are text-only without images or HTML formatting. If all the imagery and formatting is taken away, make sure your copy still gets the message across.

Dos and Don'ts:

- ✓ Make the immediate purpose of the email clear
- ✓ Ensure information and content is scannable
- ✓ Include action-oriented subject lines
- ✓ Use active voice in all CTAs
- ✓ Aim for two CTAs at the max
- ✗ Avoid leading users away from the CTA(s) with unnecessary links
- ✗ Avoid relying too heavily on imagery
- ✗ Avoid more than two CTAs
- ✗ Avoid using more than three words in CTAs
- ✗ Avoid long sentences or paragraphs

✓ Works well!	✗ Could be better
---------------	-------------------



Easy workflows for everyday tasks

Organizing your repository. Closing out old issues. Writing release notes. As we saw at GitHub Universe, GitHub Actions helps teams automate all their software workflows—including everyday tasks. Need to check off your to-do list? Get started with these actions.

- 1 Pull request label generator**
Automatically label a pull request based on its contents and committed files.
- 2 Welcoming new contributors**
Say hi to first-time contributors when they open a new issue in your repository.
- 3 Removing stale issues**
Mark and close issues after preset periods of inactivity, or any custom timeline you choose.
- 4 Website performance monitoring**
Audit URLs for performance, accessibility, progressive web apps, SEO, and more.
- 5 Release drafter**
Create draft releases and update them with new changes as pull requests are merged.

Looking for more pre-built actions to speed up your workflow? Check out our favorites.

[Explore more actions](#)

Questions about using Actions for more than CI/CD? [Let's talk](#)

Why it works:

- Immediately communicates the “why” or benefit of the email for the reader
- Follows a clear flow of information: an intro, body copy, and closing that all guides the reader to the CTA
- Includes a numbered list with icons for easy scanning
- Single CTA button that’s obvious and clickable
- CTA copy is specific and concise

Maximize innovation and minimize risk

Become an open source enterprise with GitHub

Hi <name>,

Maximum innovation, minimal risk: that's the goal of adopting open source.

By adding an open source software (OSS) component to your project, you benefit from the work of potentially thousands of developers-and also take on the risk of consuming code you didn't create.

As you bring in more code from outside your development team, your skills and approach to building software must also change.

Open source enterprises (OSE) are organizations that have struck a balance to get the most out of open source while managing the risks. An OSE recognizes OSS as essential, not optional, for modern software development.

Many organizations are already consuming open source software. It may sound tricky. But many companies are successful OSE's. In fact, according to a recent study, 99 percent almost all of proprietary software contains OSS. But, as open source increasingly forms the building blocks enterprise software, OSEs must understand and effectively manage the risks of this new supply chain.

Why did they go this route? To accelerate innovation. Because they knew that as an OSE, they would get important competitive advantages: rapid innovation, faster speed to market, and reduced costs.

[\[Learn more about what it means to be an open source enterprise\]](#)

Ready to see how GitHub supports OSE?

Request [a demo](#) or call us at +1 (877) 448-4820.

Build better software, faster with the innovative open source code and communities only found on [GitHub Enterprise](#). Trusted by half of the Fortune 50, GitHub Enterprise is the secure development platform for modern software teams.

What could be better:

- Email is multiple paragraphs
- No bullets or subheads to break up information
- Difficult to scan
- Long on mobile
- Hard to identify main CTA

Grammar and punctuation

Grammatical errors can be distracting. We use [grammar rules and best practices](#) to make sure our message—not a missing period—is what our audience remembers.

Times/dates

Include and spell out the month in dates. Only include and spell out the day of the week if it's helpful to your audience. Only abbreviate if space is limited. Do not use the “nth” format when referring to specific days.

Example: Speaker proposals are due on Friday, July 28.

In regions that use the 24-hour clock (military time), write out the time in hh:mm convention. In regions that use the 12-hour clock, use “am” and “pm” with a space before and after and specify an abbreviated time zone. When writing timezone abbreviations, use “PT” and “ET” instead of “PST” or “PDT” and “EST” or “EDT” no matter the region.

Example: Speaker proposals are due on Friday, July 28 at 10 am PT.

Acronyms

When introducing an acronym, always spell out the first full mention in body text and specify the acronym in parentheses. Then use the acronym for all subsequent mentions. Use either the acronym or full version in titles.

Example: Git is a version control system (VCS). Teams use VCSs to see how files change over time.

If your audience is familiar with an acronym, you can use it without spelling it out.

Example: API, CSS, HTML, npm, VR, VPN, CI/CD

Capitalization

Products

- Always capitalize products that are owned or created by GitHub and include “GitHub” when you mention them for the first time.

Example: Customize your workflow with GitHub Marketplace. Find the right tools for your workflow in Marketplace.

- Don't capitalize commonly-used GitHub features, like "topics" or "issues."
Example: Once you've created a pull request, you can push commits from your topic branch to add them to your existing pull request.

Octocats

- Always capitalize octocat when using it as a proper noun (writing about a specific octocat) and lowercase octocats when referring to octocats as a species or as a group.
- Always capitalize Mona, our octocat's name

Example: Mona the Octocat and her octocat friends had a party just for octocats.

Casing

- Always use sentence case for headlines, titles, and buttons.
Example: Sign up for GitHub
- Always use sentence case for subject lines, form titles, and form fields.
Example: Enter your work email
- Avoid CamelCase, except in the case of GitHub of course.

Numbers

Write out "one" and every number less than 10. Use numerals for numbers greater than nine, like 10 or 42,093.

Dashes

Use em dashes sparingly, and remember, not all mediums (like email subject lines) display em dashes.

Example: Our Student Developer Pack comes with unlimited repositories—perfect for all of those summer projects.

Use an en dash without spaces on either side for time spans and linked words.

Example: The GitHub Services Team offers classes Monday-Friday.

Periods

Use only one space after periods.

Never use periods or other punctuation in main headings or calls to action. Only use periods in subheadings or when writing multiple sentences (ex: a blog post).

For more, check out the [grammar](#) and [terminology](#) guides.

ACCESSIBILITY

Accessibility

Millions of developers build and collaborate on GitHub, from all regions, experience levels, and walks of life. The way our community uses GitHub can look very different, whether it's a mobile device, desktop, or a screen reader. We write for them all. It's the right thing to do—and as a company located in the US, [it's the law](#). So what does accessible content look like?

- **Using alt text**
Add [appropriate, contextual descriptions](#) to non-decorative images.
- **Using non-directional and non-sensory language**
Use “Check out the documentation” vs. “Read more here”
Use “Edit settings” vs. “Click the wrench icon”
Use “Refer to the screenshot describing the product” vs. “Refer to the image above”
- **Using descriptive CTAs**
Use “Sign up for the webcast” vs. “Learn more.”
- **Using headers and subheadings**
Put information in a logical order with clearly-defined sections.
- **Using plain language (no jargon please)**
Write at a 7th grade reading level and define unfamiliar terms.

More resources

- [Open source accessibility tools](#)
 - [Web Content Accessibility Guidelines \(WCAG\)](#)
 - [18F Accessibility Guide](#)
 - [WAVE Web Accessibility Evaluation Tool](#)
-